RELATÓRIO CONFIDENCIAL

Relatório de Auditoria de Segurança
Super Sistema Julli Severina

Data: 18 de setembro de 2026

Repositórios: js-frontend (Next.js 16 / React 19) e msorquestrador-jf (Spring Boot / JPA / JWT)

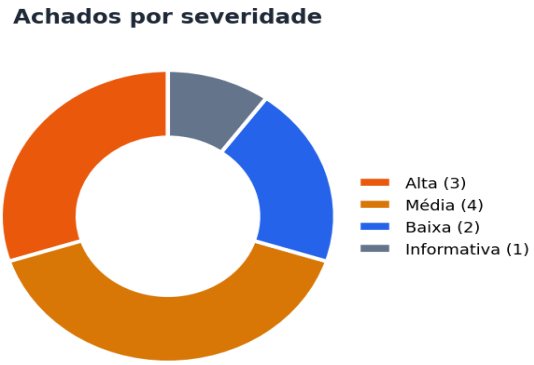
Escopo: isolamento de tenant, autorização de papel, IDOR em todos os handlers, segredos hardcoded e XSS.

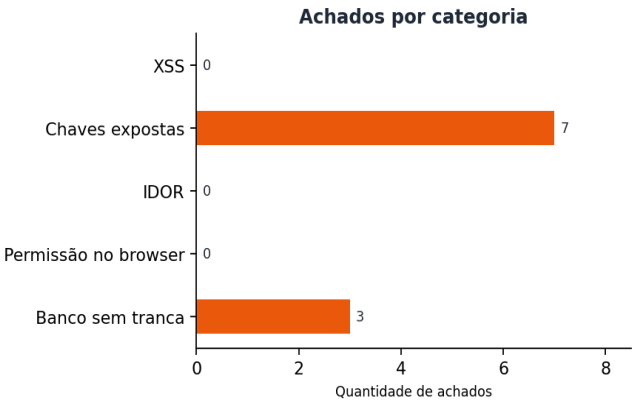
Nota metodológica. A stack não é Supabase no runtime da API: o isolamento é JWT → ClinicId no service/JPQL → RLS Postgres via `set_config('app.clinic_id')`. «Permissão no navegador» foi cruzada com `@PreAuthorize` e com a ausência de gates `isAdmin` no frontend. IDOR percorreu os 15 controllers REST (52 handlers) e as 9 rotas Next.js. Segredos incluem YAML, Compose, classes Java, `env.example` e `git log -S`. XSS cobriu `dangerouslySetInnerHTML`, TipTap, PDFs e sanitizers OWASP/DOMPurify. Só entram achados verificados no código.

1. Resumo executivo

Foram confirmados 10 achados (0 crítica, 3 alta, 4 média, 2 baixa, 1 informativa). As categorias «permissão definida no navegador», «IDOR» e «XSS» foram auditadas por completo e não produziram falhas exploráveis. O risco central está nos defaults de desenvolvimento e no endpoint público / RLS fail-open — não no CRUD autenticado.

10	3	4	3	0
Achados totais	Alta	Média	Baixa + info	IDOR / XSS





2. Pontos fortes e pontos fracos

O que está protegido

- Isolamento autenticado em três camadas. Tenant vem só do JWT (não de header). Controllers passam ClinicId.of(p.getClinicId()). JPQL filtra clinic.id (ex.: PatientJpaRepository:16–17, AppointmentJpaRepository:37–39). RLS FORCE nas tabelas tenant quando app.clinic_id está preenchido.

- IDOR: 15 controllers / 52 handlers sem falha. Todo GET/PUT/PATCH/DELETE por id (pacientes, agenda, anamnese, evolução, anexos, financeiro) usa findActiveById(..., clinicId) ou equivalente. Rotas Next.js de PDF delegam ao backend. Auth /me e /password usam p.getUserId().

- Permissão de escrita institucional no servidor. ClinicSettingsController:43 e :53 — @PreAuthorize("hasRole('ADMIN')"). O frontend não esconde a tela (navigation.ts:19–28), mas o backend recusa THERAPIST com 403. Não há gate de role no browser — a categoria «permissão só no navegador» não se aplica como falha.

- Produção recusa JWT/CPF vazios. application-prod.yml:63 allow-insecure-empty-secret: false. JwtTokenService:46–48 e CpfCryptoService:148–150 abortam o boot. scripts/security/check_config_safety.sh no CI.

- XSS tratado nas duas pontas. Único dangerouslySetInnerHTML (anamnese/page.tsx:327) passa por DOMPurify (L48–50, 55, 75). Backend: OwaspAnamneseHtmlSanitizer + AnamneseService:124 e EvolucaoService:193–200. PDF usa esc() / htmlToPlainText. Sem eval/new Function. Sem e-mail HTML.

- Sessão e superfície de API. Cookies HttpOnly + SameSite=lax (login/route.ts:90–100). CSP/HSTS em next.config.ts. Swagger desligado em prod. CORS rejeita '*'. Sem .env commitado; frontend só expõe URLs NEXT_PUBLIC_*.

Riscos centrais

- Defaults de desenvolvimento (JWT, admin, Postgres, MinIO) são segredos reais se um perfil local/dev/docker for publicado.

- O endpoint público e a policy RLS fail-open não estão prontos para multi-tenant no mesmo banco.

- Access token carrega clinic_id/role sem revalidação; mudanças manuais no DB só pegam no refresh.
- O frontend mostra Configurações a todos: THERAPIST vê o formulário e toma 403 ao salvar (UX, não bypass).

3. Tabela de achados

Chip colorido: crítica #B91C1C · alta #EA580C · média #D97706 · baixa #2563EB · informativa cinza. Detalhe arquivo-a-arquivo na secção seguinte.

Sev.	Arquivo:linha	Descrição
MÉDIA	msorquestrador-jf/.../ClinicSettingsRepositoryAdapter.java:32–33	F-01 — Endpoint público escolhe clínica arbitrária. GET /v1/public/clinic-profile (sem autenticação) usa findAll().findFirst() e expõe CNPJ, endereço, e-mail/telefone, DPO e Crefito da primeira clínica que o JPA devolver.
MÉDIA	msorquestrador-jf/.../V27__tenant_rls_policies.sql:81–87	F-02 — RLS libera todas as linhas quando app.clinic_id está vazio. A policy tenant_select permite ALL se current_setting('app.clinic_id') for nulo/vazio. Pedidos públicos não preenchem ClinicTenantContext; o datasource grava string vazia.
BAIXA	msorquestrador-jf/.../JwtAuthenticationFilter.java:87–96	F-03 — JWT não revalida clinic_id nem role contra o banco. O filtro confirma userId + tokenVersion, mas monta FisioPrincipal e ClinicTenantContext com clinic_id e role do access token, não do AppUser.
ALTA	msorquestrador-jf/.../JwtTokenService.java:51–57	F-04 — Segredo JWT de desenvolvimento hardcoded. Se JWT_SECRET estiver vazio e allow-insecure-empty-secret=true (perfis local/dev/docker), a API assina tokens com fisio-dev-jwt-secret-key-v1-abc12.
ALTA	msorquestrador-jf/.../CpfCryptoService.java:152–159	F-05 — Chave AES de CPF derivada do mesmo fallback JWT. Em não-prod, sem FISIO_CPF_ENCRYPTION_KEY e com allow-insecure-empty-secret, a chave AES-256/HMAC do CPF é SHA-256 do JWT secret ou do fallback público.
ALTA	msorquestrador-jf/.../FisioDevDataLoader.java:30,40–52	F-06 — Senha admin de seed pública no código. Perfis dev/docker criam ADMIN admin@clinica.local / Trocar@2026!dev quando app_user está vazio e FISIO_DEV_ADMIN_PASSWORD não foi definido.
MÉDIA	msorquestrador-jf/docker-compose.yml:8,24–25	F-07 — Defaults ${VAR:-teste0101} no Docker Compose. Postgres e MinIO sobem com POSTGRES_PASSWORD/MINIO_ROOT_PASSWORD=teste0101 e MINIO_ROOT_USER=minio se o .env não sobrescrever. Portas 5432/9000/9001 são publicadas no host.
MÉDIA	msorquestrador-jf/src/main/resources/application-dev.yml:6,32–33	F-08 — Senha de banco e S3 com default no YAML de desenvolvimento. DATABASE_PASSWORD:teste0101 e S3_SECRET_KEY:teste0101 / S3_ACCESS_KEY:minio nos perfis local, dev e docker.
BAIXA	msorquestrador-jf/.../FisioStorageProperties.java:31–32	F-09 — Fallback Java minio/minio123 diferente do YAML. Campos default da classe: accessKey=minio, secretKey=minio123. O YAML de dev usa teste0101; o YAML base usa string vazia.
INFORMATIVA	msorquestrador-jf/src/main/resources/application.yml:115–120	F-10 — Segredos do perfil test commitados. cpf-encryption-key Base64 fixa e jwt.secret=test-symmetric-key-32-chars-v1!! no bloco on-profile: test (H2).

4. Achados detalhados por categoria

Categoria 1 — Banco sem tranca (isolamento de inquilino)

Mecanismo detectado: não é RLS-only tipo Supabase client. A API Java é multi-tenant por clínica: claim `clinic_id` no JWT, `FisioPrincipal`, filtro JPQL e RLS Postgres. Não há isolamento por terapeuta dentro da clínica (modelo de negócio). Não há header de tenant spoofável.

F-01 · Endpoint público escolhe clínica arbitrária

MÉDIA

Arquivos
- msorquestrador-jf/src/main/java/com/mkdev/orquestrador_jf/adapters/out/persistence/ClinicSettingsRepositoryAdapter.java:32–33
- msorquestrador-jf/src/main/java/com/mkdev/orquestrador_jf/application/PublicClinicService.java:16–29
- msorquestrador-jf/src/main/java/com/mkdev/orquestrador_jf/adapters/in/web/api/PublicClinicController.java:23–26

Descrição. GET /v1/public/clinic-profile (sem autenticação) usa findAll().findFirst() e expõe CNPJ, endereço, e-mail/telefone, DPO e Crefito da primeira clínica que o JPA devolver.

Por que é explorável. Com mais de uma clínica no mesmo banco, um visitante anônimo lê dados institucionais de um tenant indefinido. A ordem de findAll() não é estável.

Condição. Explorável quando houver ≥2 linhas em clinic. Documentado como MVP single-tenant.

```
public Optional<ClinicSettings> findDefault() {
    return clinics.findAll().stream().findFirst().map(this::toDomain);
}
```

F-02 · RLS libera todas as linhas quando app.clinic_id está vazio

MÉDIA

Arquivos
- msorquestrador-jf/src/main/resources/db/migration/V27__tenant_rls_policies.sql:1–3 e 81–87
- msorquestrador-jf/src/main/java/com/mkdev/orquestrador_jf/config/TenantAwareDataSource.java:45–46
- msorquestrador-jf/src/main/java/com/mkdev/orquestrador_jf/adapters/in/web/security/JwtAuthenticationFilter.java:41–50, 55–66

Descrição. A policy tenant_select permite ALL se current_setting('app.clinic_id') for nulo/vazio. Pedidos públicos não preenchem ClinicTenantContext; o datasource grava string vazia.

Por que é explorável. Qualquer caminho JDBC sem JWT (hoje o perfil público; amanhã um job ou endpoint novo) vê e pode escrever em todas as tabelas tenant. F-01 é a prova concreta.

Condição. Postgres de produção com V27 aplicada. H2/test desliga o set_config.

```
USING (
    NULLIF(current_setting('app.clinic_id', true), '') IS NULL
    OR clinic_id::text = current_setting('app.clinic_id', true)
)
```

F-03 · JWT não revalida clinic_id nem role contra o banco

BAIXA

Arquivos

• msorquestrador-jf/src/main/java/com/mkdev/orquestrador_jf/adapters/in/web/security/JwtAuthenticationFilter.java:87–96

Descrição. O filtro confirma userId + tokenVersion, mas monta FisioPrincipal e ClinicTenantContext com clinic_id e role do access token, não do AppUser.

Por que é explorável. Um access token emitido antes de uma mudança manual de clínica/papel continua válido até expirar (15 min). Refresh já relê o banco.

Condição. Alteração de clinic_id/role no DB sem incrementar tokenVersion + access token ainda válido. Não há API admin para essa mudança.

```
var principal = new FisioPrincipal(p.userId(), p.email(), p.clinicId(), p.role());
ClinicTenantContext.set(p.clinicId());  // claims do JWT
```

Categoria 2 — Permissão definida no navegador

Não se aplica como falha. O frontend não implementa `isAdmin`, `canEdit` nem filtro de `me.role` (`lib/auth-me-api.ts:1–4` declara o campo; `lib/navigation.ts:19–28` mostra Configurações a todos). O único privilégio fino está no servidor: `@PreAuthorize("hasRole('ADMIN')")` em `ClinicSettingsController.java:43` e 53. Não existe gestão de usuários nem `DELETE /users/{id}`. Operações clínicas (delete paciente, financeiro, prontuário) são autenticadas + tenant, abertas a ADMIN e THERAPIST — comportamento verificado, sem gate invertido no browser.

Categoria 3 — IDOR

Nenhum IDOR verificado. Cobertura: Health, Auth (6), PublicClinic, Patient (7), PatientProntuario (2), PatientAttachment (2), Attachment (2), Anamnese (4), Evolucao (4), Appointment (7), Finance (8), FinancialReport (1), DashboardMetrics (1), Holiday (1), ClinicSettings (3) — 15 classes, 52 handlers. Frontend: proxy `app/api/backend/[...path]` e PDFs `app/api/pdf/prontuario/[pacienteId]` / `alta/[pacienteId]` (sessão + GET backend). Exemplo correto: `PatientController.java:61–64` → `findActiveByIdAndClinicId`; `AttachmentController.java:37–48`; `FinanceController.java:91–95`; `AnamneseController.java:42–45`.

Categoria 4 — Chaves expostas

Histórico git do backend: o literal JWT e a senha de seed existem desde `bcb8b2b` (primeiro commit da API). Nenhum `.env` tracked. Sem chaves AWS/sk-/PEM. Frontend: só URLs `NEXT_PUBLIC_*`. Produção tem validação de startup; o risco é perfil errado ou Compose sem override.

F-04 · Segredo JWT de desenvolvimento hardcoded

ALTA

Arquivos

- msorquestrador-jf/src/main/java/com/mkdev/orquestrador_jf/adapters/out/security/JwtTokenService.java:51–57
- msorquestrador-jf/src/main/resources/application-dev.yml:24–26
- msorquestrador-jf/src/main/resources/application-local.yml:13–15

Descrição. Se JWT_SECRET estiver vazio e allow-insecure-empty-secret=true (perfis local/dev/docker), a API assina tokens com fisio-dev-jwt-secret-key-v1-abc12.

Por que é explorável. Quem conhece a string (está no git desde o primeiro commit do backend) forja access tokens de qualquer usuário/clínica.

Condição. Perfil local, dev ou docker exposto sem JWT_SECRET. Produção rejeita secret vazio (application-prod.yml:63).

```
if (!props.isAllowInsecureEmptySecret()) { throw ...; }
bytes = "fisio-dev-jwt-secret-key-v1-abc12".getBytes(...);
```

F-05 · Chave AES de CPF derivada do mesmo fallback JWT

ALTA

Arquivos

- msorquestrador-jf/src/main/java/com/mkdev/orquestrador_jf/application/security/CpfCryptoService.java:148–159

Descrição. Em não-prod, sem FISIO_CPF_ENCRYPTION_KEY e com allow-insecure-empty-secret, a chave AES-256/HMAC do CPF é SHA-256 do JWT secret ou do fallback público.

Por que é explorável. CPF cifrado no banco vira reversível com uma constante do repositório.

Condição. Mesmos perfis de F-04. Em prod a chave é obrigatória.

```
String seed = StringUtils.hasText(jwtSecret) ? jwtSecret
    : "fisio-dev-jwt-secret-key-v1-abc12";
return sha256(seed.getBytes(StandardCharsets.UTF_8));
```

F-06 · Senha admin de seed pública no código

ALTA

Arquivos

- msorquestrador-jf/src/main/java/com/mkdev/orquestrador_jf/config/FisioDevDataLoader.java:30, 40–44, 52

Descrição. Perfis dev/docker criam ADMIN admin@clinica.local / Trocar@2026!dev quando app_user está vazio e FISIO_DEV_ADMIN_PASSWORD não foi definido.

Por que é explorável. Login privilegiado com credencial documentada no git e no env.example.

Condição. SPRING_PROFILES_ACTIVE=dev|docker, banco sem usuários, endpoint de login alcançável.

```
@Profile({ "dev", "docker" })
@Value("${FISIO_DEV_ADMIN_PASSWORD:}")
String pass = (...isBlank()) ? "Trocar@2026!dev" : devPassword;
```

F-07 · Defaults ${VAR:-teste0101} no Docker Compose

MÉDIA

Arquivos

• msorquestrador-jf/docker-compose.yml:8, 24–25

Descrição. Postgres e MinIO sobem com POSTGRES_PASSWORD/MINIO_ROOT_PASSWORD=teste0101 e MINIO_ROOT_USER=minio se o .env não sobrescrever. Portas 5432/9000/9001 são publicadas no host.

Por que é explorável. Credencial conhecida + bind em 0.0.0.0 típico do Docker Desktop = acesso ao banco e ao object storage na LAN.

Condição. compose sem override, host alcançável na rede.

```
POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-teste0101}
MINIO_ROOT_USER: ${MINIO_ROOT_USER:-minio}
MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD:-teste0101}
```

F-08 · Senha de banco e S3 com default no YAML de desenvolvimento

MÉDIA

Arquivos

• msorquestrador-jf/src/main/resources/application-dev.yml:6, 32–33

• msorquestrador-jf/src/main/resources/application-local.yml:8, 21–22

Descrição. DATABASE_PASSWORD:teste0101 e S3_SECRET_KEY:teste0101 / S3_ACCESS_KEY:minio nos perfis local, dev e docker.

Por que é explorável. A API autentica no Postgres/MinIO com segredo público se as env não existirem.

Condição. Perfil não-prod sem variáveis de ambiente. application-prod.yml exige DATABASE_* sem fallback.

```
password: ${DATABASE_PASSWORD:teste0101}
access-key: ${S3_ACCESS_KEY:minio}
secret-key: ${S3_SECRET_KEY:teste0101}
```

F-09 · Fallback Java minio/minio123 diferente do YAML

BAIXA

Arquivos

• msorquestrador-jf/src/main/java/com/mkdev/orquestrador_jf/config/FisioStorageProperties.java:31–32

Descrição. Campos default da classe: accessKey=minio, secretKey=minio123. O YAML de dev usa teste0101; o YAML base usa string vazia.

Por que é explorável. Se o binding YAML falhar, a app tenta MinIO com outra senha conhecida no bytecode.

Condição. Caminho raro (sem bind de fisio.storage.secret-key). Inconsistência minio123 vs teste0101.

```
private String accessKey = "minio";
private String secretKey = "minio123";
```

F-10 · Segredos do perfil test commitados

INFORMATIVA

Arquivos

• msorquestrador-jf/src/main/resources/application.yml:115–120

Descrição. cpf-encryption-key Base64 fixa e jwt.secret=test-symmetric-key-32-chars-v1!! no bloco on-profile: test (H2).

Por que é explorável. Só carrega com spring.profiles.active=test. Não é produção, mas é segredo previsível no git.

Condição. Apenas testes automatizados.

```
cpf-encryption-key: YWJjZGVmZ2hpamtsbW5vcHFyc3R1dnd4eXoxMjM0NTY=
jwt:
  secret: "test-symmetric-key-32-chars-v1!!"
```

Categoria 5 — Inputs sem tratamento (XSS)

Nenhum XSS explorável confirmado. Biblioteca presente: `isomorphic-dompurify` (frontend) e OWASP Java HTML Sanitizer (backend, `Sanitizers.BLOCKS+FORMATTING+LINKS`). Aplicação verificada no único sink HTML do React e nos create/update de anamnese/evolução. Campos de evolução na UI são texto React (escape automático). PDFs escapam com `esc()` e recusam `javascript:` em imagens (alta-html `safeImageSrc`). Não há motor de e-mail/templates no backend.

5. Recomendações priorizadas

P1. Remover literais JWT/CPF e senha admin do bytecode; exigir env em todos os perfis; lista de defaults proibidos no startup.

P1. Compose e YAML local/dev sem ${VAR:-teste0101}; falhar se a senha não vier do ambiente.

P2. Substituir findAll().findFirst() por seletor explícito de clínica pública; RLS fail-closed sem contexto.

P2. Revalidar clinic_id e role do AppUser no JwtAuthenticationFilter (ou invalidar token na divergência).

P3. Esconder a UI de Configurações para THERAPIST (alinhada ao @PreAuthorize já existente).

P3. Testes de regressão cross-tenant (clínica A acessa IDs da B → 404) e teste do endpoint público com N clínicas.

P3. Unificar fallback MinIO (apagar minio123) e placeholders CHANGE_ME no env.example.

6. Issues para o GitHub

Texto completo, pronto para colar. Achados triviais do mesmo tema foram agrupados (F-01+F-02; F-04+F-05; F-06+F-07+F-08+F-09). F-10 (perfil test) entra só como nota na issue 3 se desejarem limpar o YAML de teste. Copie o bloco entre as linhas --- ISSUE n --- e --- FIM ISSUE n ---.

Issue 1: [Segurança] Endpoint público escolhe clínica arbitrária e RLS libera tudo sem contexto

````
--- ISSUE 1 ---
# [Segurança] Endpoint público escolhe clínica arbitrária e RLS libera tudo sem contexto
**Labels sugeridas:** `security, severity:medium`
## Problema
`GET /v1/public/clinic-profile` (sem autenticação) carrega a «clínica padrão» com
`findAll().findFirst()`. Em paralelo, a policy RLS `tenant_select` permite **todas** as linhas quando
`app.clinic_id` está vazio — exatamente o estado das conexões JDBC sem JWT.
## Por que é explorável
Com duas ou mais clínicas no mesmo Postgres, um visitante anônimo recebe CNPJ, endereço, contato, DPO e
Crefito de um tenant indefinido. Qualquer código futuro que consulte tabelas tenant sem
`ClinicTenantContext` herda acesso total no banco.
## Evidência
```java
// ClinicSettingsRepositoryAdapter.java:32–33
public Optional<ClinicSettings> findDefault() {
return clinics.findAll().stream().findFirst().map(this::toDomain);
}
```
```sql
-- V27__tenant_rls_policies.sql:81–87
USING (
NULLIF(current_setting('app.clinic_id', true), '') IS NULL
OR clinic_id::text = current_setting('app.clinic_id', true)
)
```
`PublicClinicController.java:23–26` e `PublicClinicService.java:16–29`. `JwtAuthenticationFilter` não
seta contexto em `/v1/public/**`.
## Impacto
Vazamento de dados institucionais cross-tenant; defesa em profundidade do RLS inútil em paths públicos.
## Sugestão de correção
1. Trocar `findDefault()` por um seletor explícito (slug/host/`PUBLIC_CLINIC_ID`) ou 404 se houver ≠1
clínica.
2. Fail-closed no RLS quando `app.clinic_id` estiver vazio, exceto roles de migração (Flyway).
3. Não consultar tabelas tenant em requests sem `ClinicTenantContext`.
## Critérios de aceite
- [ ] Com 2 clínicas no banco, `GET /v1/public/clinic-profile` não devolve a clínica «errada» (slug
explícito ou 404).
- [ ] Policy RLS de produção **nega** SELECT em `patient`/`clinic` quando `app.clinic_id` está vazio
(testado com role da aplicação, não superuser).
- [ ] Teste de integração cobre o endpoint público com 0, 1 e N clínicas.
--- FIM ISSUE 1 ---
````

Issue 2: [Segurança] JWT de acesso não revalida clínica nem papel contra o banco

````
--- ISSUE 2 ---

# [Segurança] JWT de acesso não revalida clínica nem papel contra o banco

**Labels sugeridas:** `security, severity:low`

## Problema
`JwtAuthenticationFilter` valida `userId` e `tokenVersion`, mas monta `FisioPrincipal` e
`ClinicTenantContext` com `clinic_id` e `role` do access token.

## Por que é explorável
Se alguém alterar `app_user.clinic_id` ou `role` no banco **sem** incrementar `tokenVersion`, o access
token vigente (TTL 15 min) continua a operar na clínica/papel antigos. O refresh
(`AuthService.rotateAndIssue`) já relê o banco — o furo é só a janela do access token. Hoje não existe
API admin para essa mudança.

## Evidência
```java
// JwtAuthenticationFilter.java:87–96
var user = users.findById(p.userId());
if (user.isEmpty() || user.get().getTokenVersion() != p.tokenVersion()) { ... }
var principal = new FisioPrincipal(p.userId(), p.email(), p.clinicId(), p.role());
ClinicTenantContext.set(p.clinicId());
```

## Impacto
Acesso residual à clínica antiga ou a `@PreAuthorize("hasRole('ADMIN')")` após downgrade, até o access
token expirar.

## Sugestão de correção
Comparar `user.get().getClinic().getId()` e `user.get().getRole()` com os claims. Em divergência: 401
**ou** incrementar `tokenVersion` em qualquer UPDATE de clínica/papel.

## Critérios de aceite
- [ ] Teste: usuário com token de clínica A, `clinic_id` no DB trocado para B sem bump de versão → 401
(ou contexto B, se a decisão for «sempre DB»).
- [ ] Teste: ADMIN rebaixado a THERAPIST no DB → PUT `/v1/clinic/settings` recusado no access token
vigente.
- [ ] Qualquer mutação futura de role/clínica incrementa `tokenVersion`.

--- FIM ISSUE 2 ---
````

Issue 3: [Segurança] Segredo JWT e chave de CPF de desenvolvimento hardcoded

````
--- ISSUE 3 ---

# [Segurança] Segredo JWT e chave de CPF de desenvolvimento hardcoded

**Labels sugeridas:** `security, severity:high`

## Problema
Com `JWT_SECRET` vazio e `fisio.jwt.allow-insecure-empty-secret=true` (perfis `local`/`dev`/`docker`), a API assina JWT com `fisio-dev-jwt-secret-key-v1-abc12` e deriva a chave AES do CPF do mesmo fallback.

## Por que é explorável
A string está no git desde o primeiro commit do backend. Quem alcança uma instância nesses perfis forja tokens de qualquer usuário e, sem `FISIO_CPF_ENCRYPTION_KEY`, descriptografa CPFs.

## Evidência
```java
// JwtTokenService.java:51–57
bytes = "fisio-dev-jwt-secret-key-v1-abc12".getBytes(...);
```

```java
// CpfCryptoService.java:157–159
String seed = StringUtils.hasText(jwtSecret) ? jwtSecret : "fisio-dev-jwt-secret-key-v1-abc12";
return sha256(seed.getBytes(StandardCharsets.UTF_8));
```

`application-dev.yml:24–26` e `application-local.yml:13–15` ligam `allow-insecure-empty-secret: true`.

## Impacto
Comprometimento total da autenticação e de PII (CPF) no ambiente afetado.

## Sugestão de correção
1. Remover o literal do código; exigir `JWT_SECRET` ≥32 bytes em **todos** os perfis (gerar no README com `openssl rand -base64 48`).
2. Exigir `FISIO_CPF_ENCRYPTION_KEY` também em não-prod, ou recusar boot.
3. Fail-closed no startup se o secret for igual ao literal histórico (lista de defaults proibidos).
4. Rotacionar qualquer JWT/CPF usado em ambiente compartilhado que tenha rodado com o fallback.

## Critérios de aceite
- [ ] Grep no `src/main` não encontra `fisio-dev-jwt-secret-key-v1-abc12`.
- [ ] Boot com `JWT_SECRET` vazio falha em local/dev/docker (não só em prod).
- [ ] Boot com `FISIO_CPF_ENCRYPTION_KEY` vazio falha em não-prod, ou a derivação a partir do JWT é removida.
- [ ] Teste de startup cobre o default proibido.

--- FIM ISSUE 3 ---
````

Issue 4: [Segurança] Credenciais padrão de desenvolvimento (admin seed, Postgres, MinIO)

````
--- ISSUE 4 ---

# [Segurança] Credenciais padrão de desenvolvimento (admin seed, Postgres, MinIO)

**Labels sugeridas:** `security, severity:high`

## Problema
Vários defaults públicos viram credenciais reais se o override de ambiente faltar:

- FisioDevDataLoader.java:52 (perfis dev/docker, BD vazia): admin@clinica.local / Trocar@2026!dev
- docker-compose.yml:8,24-25: teste0101 (Postgres e MinIO), user MinIO minio
- application-dev.yml:6,32-33 e application-local.yml:8,21-22: mesma senha de banco/S3
- FisioStorageProperties.java:31-32: minio / minio123 (fallback Java, inconsistente com o YAML)

## Por que é explorável
Compose publica 5432/9000/9001. Uma API `dev`/`docker` com banco vazio cria um ADMIN conhecido. A senha está no `env.example` e no histórico git (`bcb8b2b`).

## Evidência
```java
@Profile({ "dev", "docker" })
String pass = (devPassword == null || devPassword.isBlank()) ? "Trocar@2026!dev" : devPassword;
```

```yaml
POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-teste0101}
MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD:-teste0101}
password: ${DATABASE_PASSWORD:teste0101}
```

## Impacto
Acesso administrativo à aplicação, ao Postgres (dados clínicos) e ao bucket de anexos.

## Sugestão de correção
1. Seed: falhar se `FISIO_DEV_ADMIN_PASSWORD` estiver vazio; não embutir senha no bytecode.
2. Compose: remover `:-teste0101`; exigir `.env` (Compose `required` ou script de check).
3. YAML local/dev: sem default de senha, só `${DATABASE_PASSWORD}` / `${S3_SECRET_KEY}`.
4. Apagar defaults `minio123` da classe Java.
5. Documentar geração de senhas no README; `env.example` com placeholders `CHANGE_ME`.

## Critérios de aceite
- [ ] Grep em `src/main` e `docker-compose.yml` não encontra `Trocar@2026!dev`, `teste0101`, `minio123`.
- [ ] `docker compose up` sem `.env` **falha** na falta de `POSTGRES_PASSWORD`.
- [ ] Perfil `dev` sem `FISIO_DEV_ADMIN_PASSWORD` não cria usuário (ou aborta o boot).
- [ ] `application-prod.yml` continua sem fallbacks (já está correto).

--- FIM ISSUE 4 ---
````

Fim do relatório. Gerado por docs/security-audit/generate_report.py a partir de leitura do código em 18 de setembro de 2026.
Reexecute o script após correções para atualizar gráficos e a tabela.